

[Event dashboard](#) > [Environment Setup](#) > Your Own AWS Account Setup

Your Own AWS Account Setup

If you are running this workshop on your own AWS account, you'll need to set up Python, AWS credentials, and Bedrock model access from scratch. Allow about 12 minutes.

Step 1: Set Up Your Development Environment

You'll need a code editor or IDE to write and run Python code during this workshop. Choose one of these options:

Recommended IDEs:

Visual Studio Code (Free)

- Download: <https://code.visualstudio.com/> 
- Install the Python extension for better code support
- Lightweight and excellent for Python development

JetBrains PyCharm (Community Edition - Free)

- Download: <https://www.jetbrains.com/pycharm/download/> 
- Full-featured Python IDE with excellent debugging tools
- Great for more complex development

Other Options:

- **Sublime Text** - Fast and lightweight
- **Atom** - GitHub's text editor
- **Vim/Emacs** - If you prefer terminal-based editors
- **Jupyter Notebook** - Good for interactive development



You'll be creating Python files and running them from the command line, so make sure your chosen editor can handle Python syntax highlighting and has terminal access.

Step 2: Ensure Python is Installed

 On macOS, you may need to use `python3` and `pip3` instead of `python` and `pip`. You can create aliases to simplify this, by running these commands on your IDE Terminal: `alias python='python3' && alias pip='pip3'`

In your IDE terminal, run these commands:

```
1 # Check Python version (should be 3.7 or higher)
2 python --version
```



If Python is not installed, visit: <https://python.org/downloads> 

Step 3: Set Up AWS Credentials

Choose one of these methods:

Option A: AWS CLI (Recommended)

In your IDE terminal, run these commands:

```
1 # Install AWS CLI if not already installed
2 # Visit: https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html
3
4 # Configure your credentials
5 aws configure
```



When prompted, enter:

- **AWS Access Key ID:** Your access key
- **AWS Secret Access Key:** Your secret key
- **Default region name:** `us-west-2`
- **Default output format:** `json`

Option B: Environment Variables

In your IDE terminal, run these commands:

```
1 export AWS_ACCESS_KEY_ID=your_access_key_here
2 export AWS_SECRET_ACCESS_KEY=your_secret_key_here
3 export AWS_DEFAULT_REGION=us-west-2
```



Step 4: Enable Claude 4.5 Sonnet in Amazon Bedrock

Make sure the IAM role of the CLI user above has appropriate permissions to invoke Claude 4.5 Sonnet, refer this [Access Amazon Bedrock foundation models](#) 

 Without the correct IAM permissions, your agent won't work! This is a critical step.

Step 5: Create Project Directory

In your IDE terminal, run these commands:

```
1 # Create and navigate to project directory
2 mkdir agentic-ai-workshop
3 cd agentic-ai-workshop
4
5 # Create Python virtual environment
6 python -m venv .venv
7 # or if python command doesn't work:
8 # python -m venv .venv
9
10 # Activate virtual environment
11 # On macOS/Linux:
12 source .venv/bin/activate
13 # On Windows:
14 # .venv\Scripts\activate
```



You should see `(.venv)` in your terminal prompt indicating the virtual environment is active.

Step 6: Install Dependencies

In your IDE terminal, run these commands:

```
1 # Install required packages
2 pip install boto3>=1.34.0 streamlit>=1.28.0 requests>=2.31.0 Pillow>=10.0.0
```



This will take a minute or two to download and install all the packages.

Step 7: Verify Your Setup

In your IDE terminal, run these commands:

```
1 # Test AWS connection
2 aws sts get-caller-identity
3
4 # Test Python imports
5 python -c "import boto3, streamlit, requests; print('✅ All packages installed')"
```



Troubleshooting Common Issues

"AWS credentials not found"

- Run `aws configure` and enter your credentials
- Or set environment variables as shown in Step 2

"Model access denied"

- Go to Bedrock console → Model Access → Request access for Claude 4.5 Sonnet
- Make sure you're in the us-west-2 region
- Wait a few minutes and try again

"Python command not found"

- Try using `python` instead of `python`

"Permission denied" errors

- Ensure your AWS user has the following permissions:
 - `bedrock:InvokeModel`
 - `bedrock:Converse`
 - `bedrock:ListFoundationModels`

Virtual environment issues

In your IDE terminal, run these commands:

```
1 # If activation fails, try:
2 python -m venv --clear .venv
3 source .venv/bin/activate
```



Package installation fails

In your IDE terminal, run these commands:

```
1 # Try upgrading pip first
2 pip install --upgrade pip
3 pip install boto3>=1.34.0 streamlit>=1.28.0 requests>=2.31.0 Pillow>=10.0.0
```



Required AWS Permissions

Your AWS user needs these minimum permissions:

```
1 {
2     "Version": "2012-10-17",
3     "Statement": [
4         {
5             "Effect": "Allow",
```



```
6         "Action": [  
7             "bedrock:InvokeModel",  
8             "bedrock:Converse",  
9             "bedrock:ListFoundationModels",  
10            "bedrock:GetFoundationModel"  
11        ],  
12        "Resource": "*"   
13    },  
14    {  
15        "Effect": "Allow",  
16        "Action": [  
17            "sts:GetCallerIdentity"  
18        ],  
19        "Resource": "*"   
20    }  
21 ]  
22 }
```

Setup Complete

You should now have:

- Python 3.7+ installed and working
- Python virtual environment activated (you'll see `(.venv)` in your prompt)
- All required packages installed
- AWS credentials configured and tested
- Claude 4.5 Sonnet access enabled in Bedrock
- A working directory called `agentic-ai-workshop`

Your environment is ready. Next we'll look at how the agent's architecture fits together.

Next: Understanding the Architecture

[Previous](#)[Next](#)

oscarrodriguezwork ▼